INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
MINAS GERAIS
Campus Bambuí

Ciência da Computação

Trabalho Prático

Programação Orientada a objetos

Disciplina: Programação Orientada a objetos Data: ___/___/_____

Professor: Bruno Ferreira Turma: T1 e T2

Aluno Valor: Nota: 28,0

Objetivo: Reforçar conceitos básicos sobre Orientação a objetos, além de vivenciar o desenvolvimento de um sistema com mais requisitos.

Forma de Entrega: O código fonte zipado deverá ser entregue via Google Class e Google Drive. O trabalho é em dupla, mas a APRESENTAÇÃO É INDIVIDUAL, apenas um aluno envia o trabalho com o nome dos dois alunos. O trabalho será divido em três etapas:

Versão Inicial (05/11/2026 até as 23:59) – 10,00 pontos

Especificação inicial (Classes + pacotes + interface visual)

Versão Dois (26/11/2026 até as 23:59) – 9,00 pontos

Camada de Serviço + Herança + Enumerator

Versão Final (09/12/2026 até as 23:59) – 9,00 pontos

O sistema deverá persistir os dados em um banco dados relacional ou de documentos.

A apresentação será nos dias **10 e 14/12/2026**. O envio dos arquivos deverão ser feitas conforme cronograma acima.

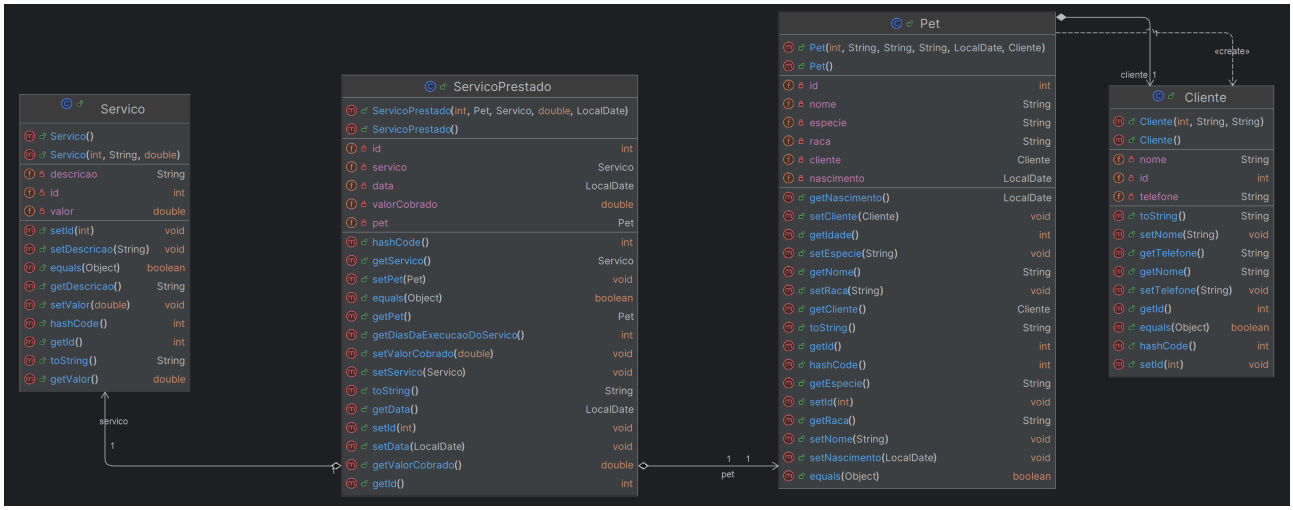
Durante a apresentação, os

OBS: Será observado e avaliado a legibilidade do código, sendo fundamental uma boa endentação e a utilização de comentários.

Se for verificada cópia de trabalho, os trabalhos serão considerados com nota zero.

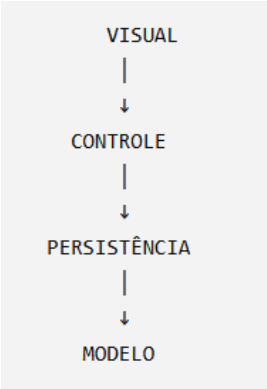
Vamos criar um protótipo de sistema para gerenciar um petshop, o famoso **Petshop Bichin**. O sistema terá o cadastro de serviços, clientes, animais (pet) e registrará os serviços prestados aos bichos. Vamos vincular o serviço prestado diretamente ao pet e não ao cliente. Podemos analisar essa modelagem na camada/pacote de modelo (model) do diagrama de classe.

Passo 1) Crie o pacote *model* e implemente as classes conforme o diagrama abaixo. Repare que as classes Pet e ServicoPrestado têm um método para calcular períodos de data. Lembre-se que o construtor instância objetos e estruturas de dados internas.



Repare as associações entre as classes. Um serviço e um pet existem sem o ServiçoPrestado (agregação), mas o "serviço prestado" precisa destes dois objetos, já um Pet não exite sem um Cliente (composição). Uma associação simples com o esterótipo «create» indica que a classe Pet cria um objeto interno do tipo Cliente, provavelmente no construtor.

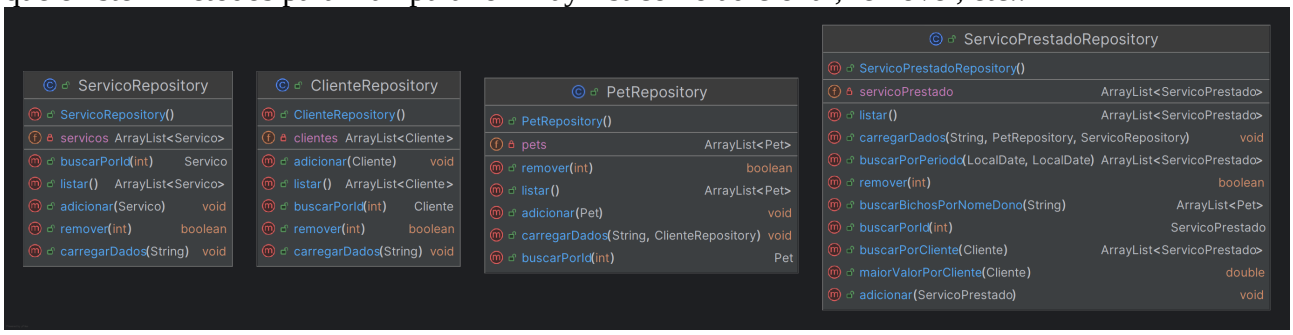
A arquitetura interna das camadas/pacotes terão uma estrutura semelhante a imagem abaixo:



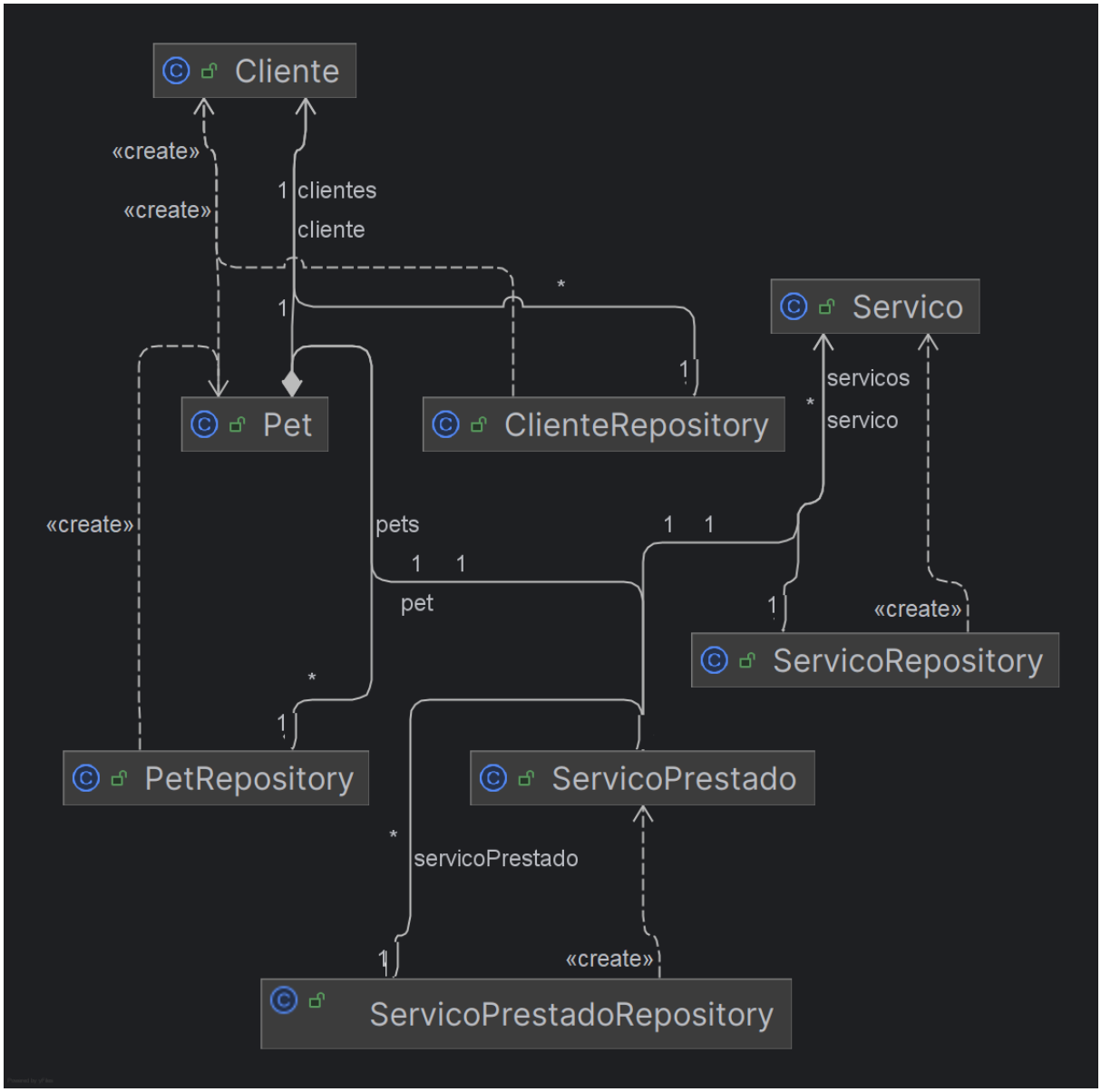
A ideia é que possamos mudar uma camada sem ter que alterar as outras. Por exemplo, podemos mudar o visual de "linha de comando" para uma interface gráfica ou mudar a persistência de dados de vetor, para Set, ou mesmo, para um banco de dados relacional sem afetar a camada de visual.

Para essa primeira versão, o trabalho usará Arraylist para armazenar os dados do projeto, as classes de repository serão responsáveis por persistir os dados e terão métodos para buscar dados iniciais de arquivos do tipo texto, o método "carregar dados" recebe o caminho e nome do arquivo com as informações que serão inseridas na aplicação.

Passo 2) Implemente a camada de persistência (repository) conforme o diagrama abaixo, observe que existem métodos para manipular o ArrayList como adicionar, remover, etc.:



Para uma visão geral dos relacionamentos até o momento, veja o diagrama abaixo (as propriedades e ações foram ocultadas):



O esterótipo «create» indica que internamente a classe cria um objeto do tipo indicado, Por exemplo, no método carregaArquivo, da classe PetRepository, é instanciado um objeto do tipo Pet, essa é uma associação entre classes da forma mais fraca, semelhante à “delegação”. Mais uma vez, veja no diagrama a associação forte chamada composição (◆) entre Pet e Cliente, ou seja, no

sistema não faz sentido ter um cliente se não tem um Pet associado a ele. Semanticamente poderíamos tratar isso no sistema não deixando criar um cliente se não foi informado o Pet desse cliente. As outras associações são simples e expressam uma troca de mensagem, ou podem ser parâmetros de entradas em alguns métodos ou indicação que serão usadas na estrutura de dados: ArrayList<Servico>.

A próxima camada a ser desenvolvida será a de controle (controller). Ela vai conectar a camada de visão (interface com o usuário) com a camada de persistência de dados (repository). Essa camada será responsável por permitir a substituição de uma camada sem afetar a outra. A mágica é que essa classe recebe o **repository como parâmetro no construtor** e tem praticamente os mesmos métodos que um objeto desse repository recebido como parâmetro de entrada. A camada de controller simplesmente chama os métodos do repository, Veja exemplo abaixo:

```
public class PetController {

   private PetRepository repository;
   private ClienteRepository repositoryCliente;

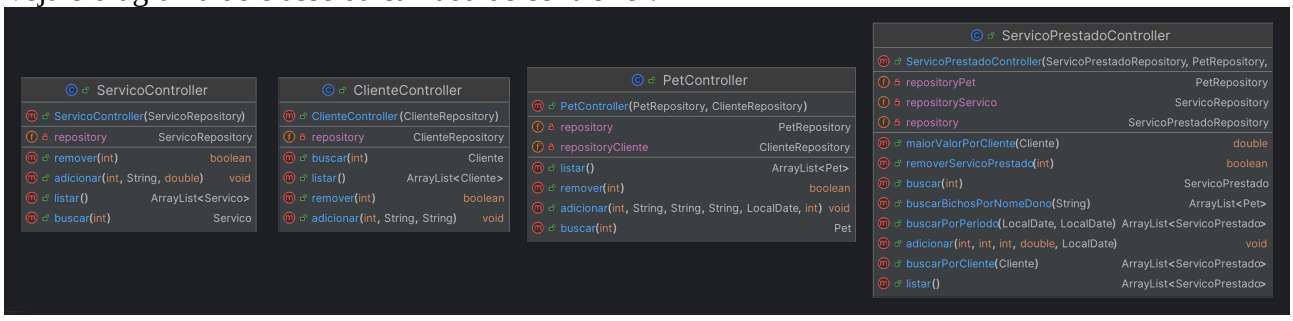
   public PetController(PetRepository repository, ClienteRepository repositoryCliente) {
      this.repository = repository;
      this.repositoryCliente = repositoryCliente;
   }

   public Pet buscar(int id) {
      return repository.buscarPorId(id);
   }

   public ArrayList<Pet> listar() {
      return repository.listar();
   }
  // restante dos métodos omitidos...
}
```

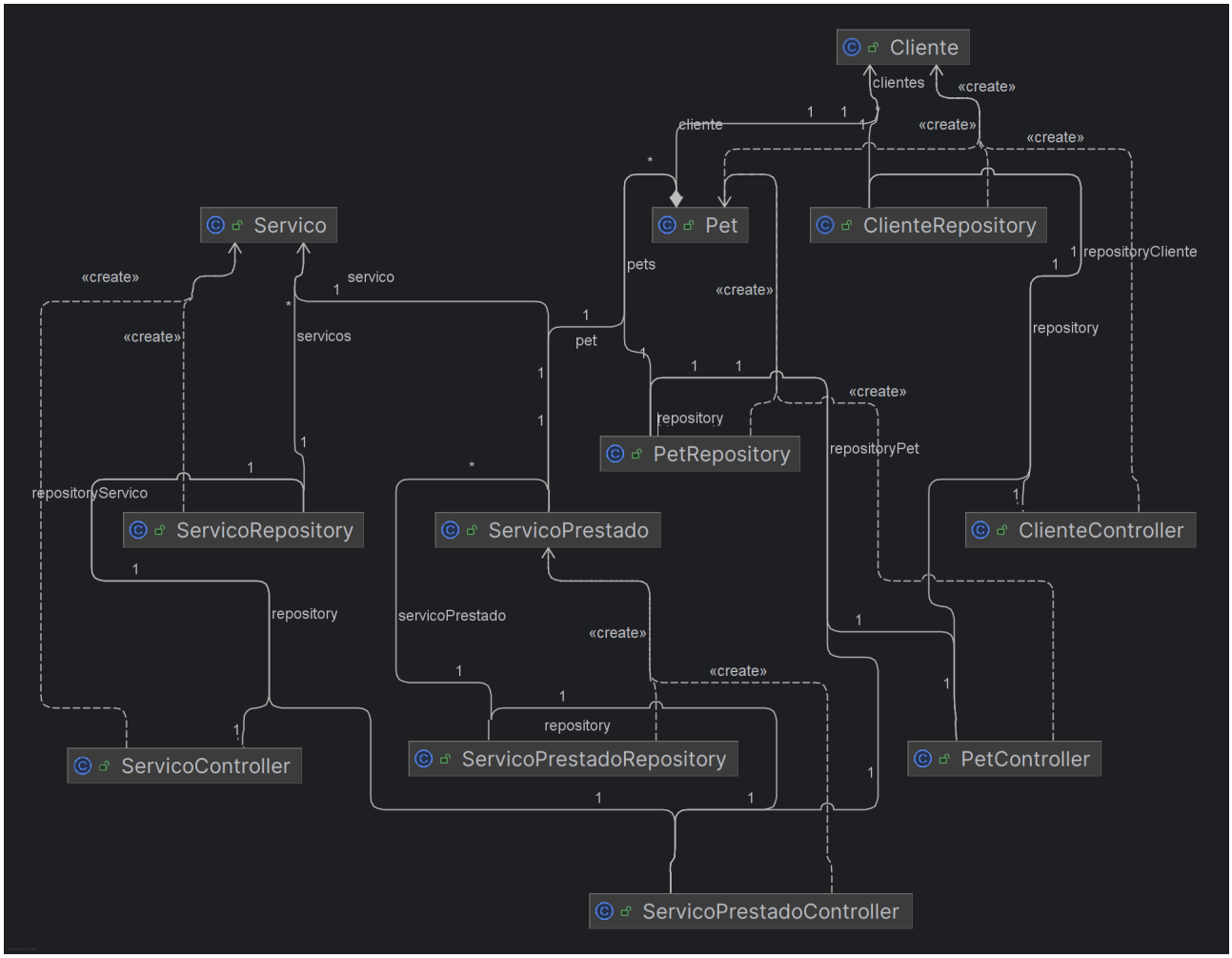
Nesse código acima está o segredo, você pode modificar o PetRepository repository para acessar o banco de dados, pois a camada de visão, a qual acessa a camada de controller, não vai ver essa mudança, esse é o poder do encapsulamento, a boa organização de camadas (pacotes) e classes coesas e fracamente acopladas.

Veja o diagrama de classe da camada de controller:



Agora, para ter uma visão geral de como nosso projeto está crescendo, veja o diagrama de classe com o foco no relacionamento entra as classes nos três pacotes, lembre-se que o esterótipo «create» diz que uma classe instância objetos internos, a composição serve para demonstra que um objeto não faz sentido sem o outro, a associação simples diz que uma classe tem como propriedade outra classe, já a delegação mostra que um método da classe tem como parâmetro o objeto de outra classe, ou seja, faz referência a outra classe:

Em Orientação a Objetos, um objeto pode enviar uma mensagem para outro objeto solicitando que ele execute um determinado comportamento, normalmente por meio da chamada de um método.



Passo 3) Finalmente, vamos avançar para a camada de visão. O projeto terá o pacote `visao.cmd`; ou seja, dentro do pacote `visao` teremos o pacote `cmd`, responsável pela interface em linha de comando. Inicialmente, nossa aplicação utilizará essa interface. Futuramente, caso desejarmos modificar o projeto para utilizar uma interface gráfica com o usuário (GUI — *Graphical User Interface*), poderemos criar o pacote `gui` dentro de `visao`. Dessa forma, a alteração da interface não exigirá mudanças significativas nas demais camadas do sistema.

A transparência das modificações está na forma como as camadas se comunicam, especialmente por meio das classes do pacote `controller`. As classes responsáveis pela interface com o usuário receberão as classes da camada `controller` como parâmetros em seus construtores (chamamos isso de injeção de classes/dependencia). Dessa forma, a interface gráfica apenas precisará chamar os métodos disponibilizados pelos `controllers`, sem precisar conhecer os detalhes das demais camadas.

Se futuramente substituirmos a interface gráfica por outro tipo de interface, basta seguir a mesma regra: a nova classe de interface receberá os `controllers` necessários em seu construtor e utilizará seus métodos para executar as operações do sistema. Assim, podemos modificar a interface sem precisar alterar as regras de negócio já implementadas.

Observe no código abaixo que a interface do usuário responsável pelo cadastro de Pets fica em uma classe exclusiva. Ela recebe os controladores de **Cliente** e **Pet**, permitindo que um novo cliente seja cadastrado simultaneamente ao registrar um animal.

```
public class UiCmdPet { //Classe de iteração com o usuário para as opções do Pet (animal).

   private Scanner teclado;

   private PetController controller;
   private ClienteController controllerCliente;

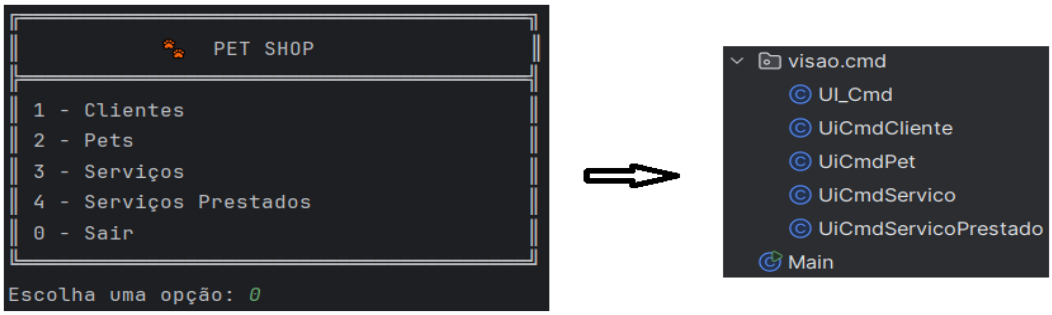
   public UiCmdPet(
         PetController controller,
         ClienteController controllerCliente) {

      teclado = new Scanner(System.in);

      this.controller = controller;
      this.controllerCliente = controllerCliente;
   }

   //demais métodos da classe...
}
```

Por questões de manutenibilidade. Aqui segue apenas uma sugestão para implementara a camada de visão: Separar cada item do menu principal em uma classe:



A classe **UI_Cmd** tem o menu principal, o qual vai chamar métodos dos objetos que representam as opções do menus. Exemplo: ao escolher o cadastro de serviços, o sistema chama o método “iniciar()” do objeto de interface gráfica do tipo UICmdServico, ao escolher o cadastro de pets, o sistema chama o método “iniciar()” de um objeto do tipo UICmdPet. A classe UI_Cmd recebe as opções de menu, veja um trecho do código:

```
public class UI_Cmd {

 private Scanner teclado;

 private UiCmdCliente uiCmdCliente;
 private UiCmdPet uiCmdPet;
 private UiCmdServico uiCmdServico;
 private UiCmdServicoPrestado menuServicoPrestado;

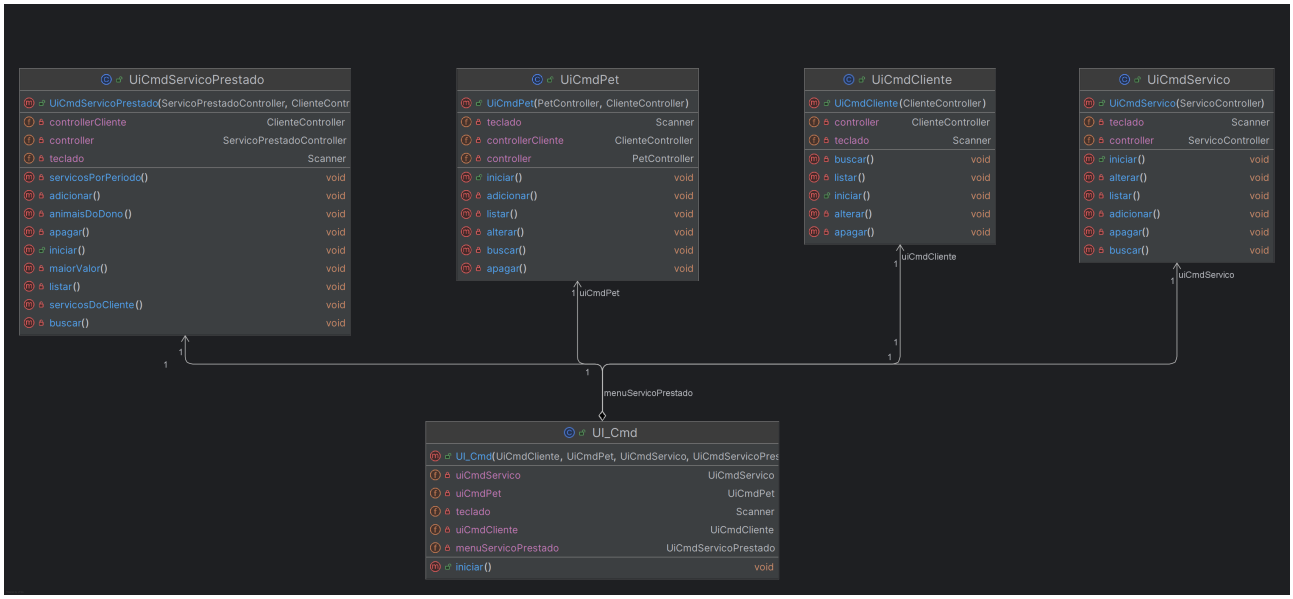
 public UI_Cmd(
    UiCmdCliente uiCmdCliente,
    UiCmdPet uiCmdPet,
    UiCmdServico uiCmdServico,
    UiCmdServicoPrestado menuServicoPrestado) {

          teclado = new Scanner(System.in);
```

```
        this.uiCmdCliente = uiCmdCliente;
        this.uiCmdPet = uiCmdPet;
        this.uiCmdServico = uiCmdServico;
        this.menuServicoPrestado = menuServicoPrestado;
    }

    public void iniciar() {
        //método que mostra o menu de opções em um laço de repetição,
        //lê a opção desejada pelo usuário e chama o método inciar() do objeto que representa a opção escolhido
    }
}
```

Para ajudar, nessa sugestão de implementação, segue o diagrama de classe do pacote visao.cmd:



Veja a sutileza do relacionamento da **associação por agregação** entre U_Cmd e as outras classes, essa relação diz que existem um papel de “todo” e “parte”, mas o “todo” e a “parte” podem viver uma sem a outra, Nesse caso, as classes UiCmdServicoPrestado, UiCmdPet, UiCmdCliente e UiCmdServico podem ser instanciadas sem a UI_Cmd, mas essa última classe, só tem sentido em existe, se informarmos todas as outras classes no construtor, as quais são as ações do menu.

Conforme apresentado no diagrama, o método público `iniciar()` é o único exposto externamente, sendo responsável por exibir o menu da opções escolhida. Todas as operações selecionadas são processadas por métodos privados da própria classe. Por exemplo, ao optar por listar os serviços oferecidos, o menu chama o método privado `listar()`, que por sua vez solicita a execução à camada *Controller.*

Talvez, a implementação da camada de visão fique mais visível ao entender como é implementado o método main:

```
package br.edu.ifmg.poo04;

import br.edu.ifmg.poo04.controller.*;
import br.edu.ifmg.poo04.model.*;
import br.edu.ifmg.poo04.repository.*;
import br.edu.ifmg.poo04.visao.cmd.*;
```

```java
public class Main {

    public static void main(String[] args) {

        /*
         * ========================================
         * AQUI  DEVEMOS CRIAR OS OBJETOS DAS CLASSES DA CAMADA REPOSITORIES
         *Lembre-se, essa camada representa a persistência dos dados.
         *As classes desse nível serão injetadas na camada de controller
         * ========================================
        */



        /*
         * ========================================
         * CARREGAMENTO DOS DADOS INICIAIS
         * Aqui são chamados os métodos de carregamento de dados, a partir dos aquivos texto.
         * Exemplo: clienteRepository.carregarDados("dados/clientes.txt");
         * ========================================
         */

        /*
         * ========================================
         * AQUI  DEVEMOS CRIAR OS OBJETOS DAS CLASSES DA CAMADA CONTROLLERS
         *Lembre-se, essa camada representa o objeto que vai intermediar a comunicação entre visão e repository.
         *Os construtores das classes, dessa camada, recebem o objeto que salva os dados, Arraylist, BD, arquivos.
         *Logo abaixo, as classes dessa camada serão injetadas nas classes da camada de visão.
         * ========================================
         */

        /*
         * ========================================
         * Cria cada classe da interface do usuário informando a classe de controller que ela deve manipular (MENUS)
         * Exemplo: UiCmdCliente uiCmdCliente =  new UiCmdCliente( clienteController );
         * ========================================
         */

        /*
         * ========================================
         * Por fim, criação da classe que recebe as opções do menu e exibe os dados: MENU PRINCIPAL
         * ========================================
         */

        UI_Cmd UICmd =
                new UI_Cmd(
                        uiCmdCliente,
                        uiCmdPet,
                        uiCmdServico,
                        menuServicoPrestado
                );

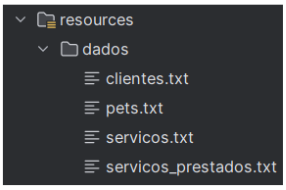
        UICmd.iniciar();

    }
}
```

Talvez com o main pronto, fique mais visível a questão da manutenção do sistema. 1) Trocando as classes do repository, a classe de visão não fica sabendo da mudança, 2) se trocarmos as classes de visão, a camada de repository também não fica sabendo da mudança. Assim, pode-se trocar a interface gráfica ou o modo de persistência de dados, de forma transparente ao resto do sistema.

Algumas regras:

- Como um Pet não pode existir sem um cliente (relacionamento de composição), o cliente só pode ser criado no momento da inserção do Pet. Assim, o sistema deve pedir o código do cliente, se ele existir, o sistema associa o cliente ao animal, se a pessoa não existir ela deve ser criada nesse momento.
- Datas devem ser válidas e estar no formato brasileiro. Trate a exceção de conversão de data e leia a entrada do usuário até que ele informe o dado correto. Padrão dd/MM/yyyy.
- Telefones devem estar no formato correto: (dd) ddddd-ddddd. Trate a exceção de formato de telefone e leia a entrada do usuário até que ele informae o dado correto.
- No valor cobrado pelo usuário, ao cadastrar um serviço prestado, o sistema deve mostrar o valor e o serviço escolhido, antes do usuário digitar o valor efetivamente cobrado.
- Ao iniciar o sistema, preencha o Arraylist com os dados vindos de arquivos textos.



Exemplo do arquivo de cliente.

```
id;nome;telefone
1;João da Silva;99999-1111
2;Maria Oliveira;99999-2222
3;Carlos Souza;99999-3333
4;Ana Paula;99999-4444
5;Pedro Santos;99999-5555
```

Versão 2

1) Muitas das vezes, os sistemas têm regras de negócio complexas. Então surge a dúvida, aonde devemos implementar essas regras? Uma opção de projeto, nesse caso, é criar mais uma camada chamada “service”. Essa camada não precisa existir para as classes que não tem regras complexas. Para nosso protótipo, ao prestar um serviço, o sistema tem duas regras:

1. O usuário não pode cadastrar um serviço prestado no passado;
2. O usuário não pode informar um valor negativo na prestação do serviço.

Então o sistema terá uma nova camada (service), ela será necessária apenas ao domínio do negócio que trata do “ServicoPrestado”:

Main
- cria Repository “ServicoPrestado”, o qual será injetado na camada abaixo
- cria o ServicoPrestadoService, o qual será injetado na camada abaixo
  - recebe ServicoPrestadoRepository
- cria ServicoPrestadoController, o qual será injetado na camada de visão
  - recebe ServicoPrestadoService

Isso é interessante, porque cada classe passa a ter uma responsabilidade clara:

Main: Monta o sistema e conecta os objetos.

Controller: Recebe as solicitações da interface e chama o Service.

Service: Executa as regras de negócio e chama o Repository.

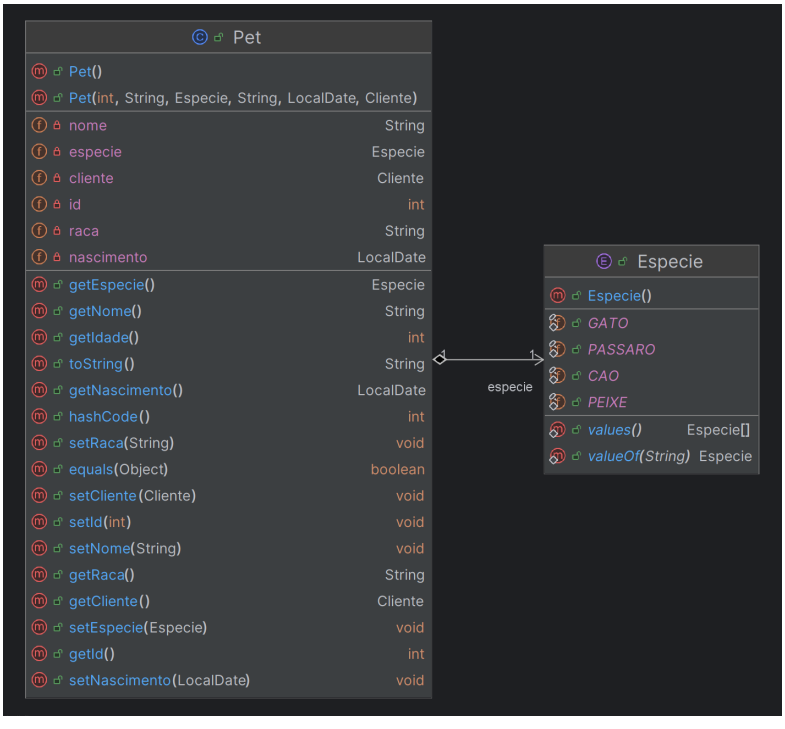
Repository: Armazena e consulta os dados.

Visão: Mostra os elementos de interação com o usuário. Veja o diagrama abaixo e repare nas associações e nos construtores. Esse diagrama mostra um recorte do sistema responsável pelo domínio dos serviços prestados pelo petshop.

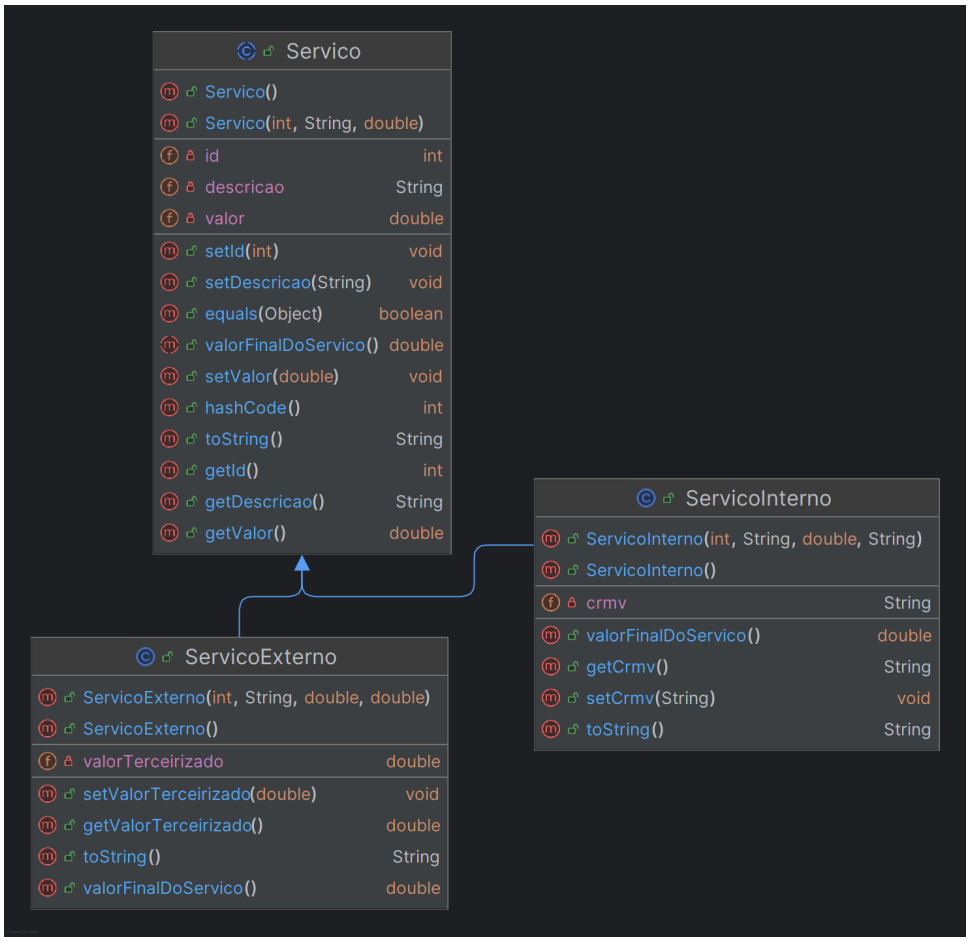
Alguns desenvolvedores criam a camada de serviço, mesmo se não existem regras de negócio complexas, pois se em um futuro próximo, os requisitos exigirem isso, o projeto já está estruturado para a nova demadna. Quando não existem regras de negócio, basta chamar os métodos do repository sem criar código extra.



2) A segunda melhoria será aceitar a espécie dos animais com valores fixos, assim não existem problemas com erros de digitação, o petshop aceita apenas animais de pequeno porte, mais especificamente: CÃO, GATO, PÁSSARO e PEIXE. Uma ótima solução para isso, é usar um enumarator, contudo, faça as alterações necessárias na camada de visão e model, nessa última, crie a Especie.java e modifique a propriedade na classe Pet de String para esse novo enum.



3) Por último, vamos implementar diferentes tipos de serviços prestados, existem os serviços **internos** e os serviços **externos**, a diferença é que esse último conta com um valor extra, pois depende de profissionais terceirizados, por exemplo, acupuntura para cachorros. Nesse caso, o valor é calculado pelo valor-base + o valor do profissional, para serviços internos é usado apenas o valor-base. O serviço interno também deve armazenar o CRMV (conselho regional de medicina veterinária), o qual representa a pessoal responsável pelo procedimento. Uma modelagem interessante seria usar herança, deixando a classe Serviço como abstrata, inserido assim, o método valorFinalDoServico para ser implementando pelas classes especializadas (Interno e Externo). Essas classes filhas teriam as propriedades CRMV e "valor do terceirizado" como diferença.



Na camada de visão, a classe UiCmdServico. terá que requisitar qual o tipo de serviço, conforme mostra o código baixo, veja no final que agora passamos o CRMV e o valor do terceirizado no método "adicionar" da camada de controller:

```
System.out.print("Qual o tipo do serviço: 1)Interno ou 2) Externo");
int tipo = teclado.nextInt();

String responsavel=null;
double valorTerceirizado=0;
if (tipo == 1) {
    teclado.nextLine();
   System.out.print("CRMV do responsável pelo  serviço: ");
   responsavel = teclado.nextLine();
}
else {
   System.out.print("Valor do serviço terceirizado: ");
   valorTerceirizado = teclado.nextDouble();
}
```

```
 controller.adicionar(
       id,
       descricao,
       valor,
       responsavel,
       valorTerceirizado
  );
//...
```

No controlador instanciamos o objeto correto de acordo com os dados enviados:

```
//...
Servico servico;

if (responsável == null || responsável.isBlank()) {
   servico = new ServicoExterno(
          id,
          descricao,
          valor,
          valorTerceirizado
   );
} else {
   servico = new ServicoInterno(
          id,
          descricao,
          valor,
          responsável
   );
}
//...
```

Versão 3

Na última versão, vamos persistir os dados em um banco de dados relacional ou em um BD de documentos. Criar exceções e tratar os erros da melhor forma possível.

Como separamos **Visual → Controller → Service → Repository**, essa é justamente a hora em que a arquitetura começa a mostrar uma vantagem importante: **a maior parte da aplicação pode permanecer igual**.

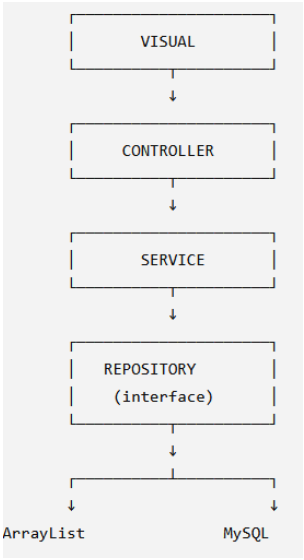
A principal mudança da versão 3 será na **camada de persistência**.

Visual	Visual
↓	↓
Controller	Controller
↓	↓
Service	Service
↓	↓
Repository	Repository
↓	↓
ArrayList	MySQL

Antes x Depois

Ou seja, o `Service`, `Controller` e `Visual` não deveriam precisar saber se os dados estão em um `ArrayList`, arquivo `.txt` ou MySQL.

Vamos fazer uma melhoria, que não é obrigatória, mas deixará o nosso projeto mais profissional. Ou seja, criar uma interface para representar a camada de repository, depois cada método de armazenamento assinará essa interface implementado o método de persistência escolhido.



A interface vai facilitar a mudança do tipo de armazenamento no main, veja o exemplo abaixo, quando declaramos um objeto da camada de repository, o fazemos pelo nome da interface e não da classe, assim, quando desejamos modificar de um tipo de armazenamento para outro, é só mudar a instrução com a palavra new seguida pela chamada do construtor. Veja a declaração e criação para ArrayList e MySql

```
//Declaramos o repository pela interface a qual foi nomeada de “Repository”
Repository clienteRepository;
Repository servicoRepository;
Repository petRepository;
Repository servicoPrestadoRepository;

//Criamos o repository usando os ArrayList
clienteRepository =  new ClienteRepository();
servicoRepository = new ServicoRepository();
petRepository = new PetRepository();
clienteRepository = new ClienteRepository();

//Ou podemos criar o repository usando o BD
clienteRepository =  new ClienteRepositoryMySql();
servicoRepository = new ServicoRepositoryMySql();
petRepository = new PetRepositoryMySql();
clienteRepository = new ClienteRepositoryMySql();
```

Todo o código fonte que fazia declarações usando ClienteRepository, ServicoRepository, PetRepository ou ClienteRepository podem ser trocadas pela interface, exemplo, na classe da camada de controle:

```
ublic class ClienteController {

   private Repository<Cliente> repository;

   public ClienteController(Repository<Cliente> repository) {
      this.repository = repository;
   }
   //código restante omitido e sem alteração...
}
```

Vamos ter que alterar também a camada de serviço, pois ela ainda não faz referência a interface:

```
public class ServicoPrestadoService {

    private Repository<ServicoPrestado> repository;

    public ServicoPrestadoService(Repository<ServicoPrestado> repository) {
        this.repository = repository;
    }
     //código restante omitido e sem alteração...
}
```

Então como ficaria a interface? Vamos usar o conceito de Generics para criar a interface na camada/pacote de repository:

```
package br.edu.ifmg.poo04.repository;

import java.util.ArrayList;

public interface Repository<T> {

    void adicionar(T objeto);

    T buscarPorId(int id);

    ArrayList<T> listar();

    boolean alterar(int id, T objeto);

    boolean remover(int id);
}
```

Agora todos os repositories devem assinar essa interface, veja o exemplo para o domínio do Cliente:

```
public class ClienteRepository implements Repository<Cliente> {

  //demais código não mudam, pois o nome dos métodos já coincidiam...

}
```

Lembre-se: A interface funciona como um **contrato**. Ela define quais operações um repositório deve oferecer, mas não define como essas operações serão realizadas.

A interface permite que o restante do sistema trabalhe com um contrato, e não com uma implementação específica. Agora que está pronto, podemos trocar a forma de armazenamento dos dados para outra opção, como MongoDB, alterando principalmente a implementação escolhida no `Main.` Mas antes de fazer isso, vamos finalizar a implementação do MySQL.

Como estamos trabalhando com projeto Maven, insira essa dependência no arquivo pom.xml:

```
<!-- Source: https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.33</version>
    <scope>compile</scope>
</dependency>
```

Essa tag <dependency> fica dentro de uma tag <dependencies>.

Depois crie a classe de conexão usando o padrão Singletom, conforme abaixo (repare que criei um pacote dentro de repository chamado mysql:

```
package br.edu.ifmg.poo04.repository.mysql;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Conexao {

    private static Connection conexao;

    private Conexao() {
    }

    public static Connection getConexao() {

        try {
            if (conexao == null || conexao.isClosed()) {

                try {

                    String url = "jdbc:mysql://localhost:3306/petshop";
                    String usuario = "root";
                    String senha = "root";

                    conexao = DriverManager.getConnection(
                            url,
                            usuario,
                            senha
                    );

                    System.out.println("Conectado ao MySQL!");

                } catch (SQLException e) {

                    throw new RuntimeException(
                            "Erro ao conectar ao banco de dados.",
                            e
                    );
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }

        return conexao;
    }
}
```

Agora é só finalizar criando os repositórios que acessam o banco de dados, veja um exemplo:

```
package br.edu.ifmg.poo04.repository.mysql;

import br.edu.ifmg.poo04.model.Cliente;
import br.edu.ifmg.poo04.repository.Repository;

import java.io.BufferedReader;
import java.io.InputStream;
```

```java
import java.io.InputStreamReader;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

public class ClienteRepository implements Repository<Cliente> {


    @Override
    public void adicionar(Cliente cliente) {

        String sql = """
                INSERT INTO cliente
                (id, nome, telefone)
                VALUES (?, ?, ?)
                """;

        try (
                Connection conexao = Conexao.getConexao();
                PreparedStatement comando =
                        conexao.prepareStatement(sql)
        ) {

            comando.setInt(1, cliente.getId());
            comando.setString(2, cliente.getNome());
            comando.setString(3, cliente.getTelefone());

            comando.executeUpdate();

        } catch (SQLException e) {

            throw new RuntimeException(
                    "Erro ao inserir cliente.",
                    e
            );
        }
    }

    @Override
    public Cliente buscarPorId(int id) {

        String sql = """
                SELECT id, nome, telefone
                FROM cliente
                WHERE id = ?
                """;

        try (
                Connection conexao = Conexao.getConexao();
                PreparedStatement comando =
                        conexao.prepareStatement(sql)
        ) {

            comando.setInt(1, id);

            ResultSet resultado = comando.executeQuery();

            if (resultado.next()) {

                return new Cliente(
                        resultado.getInt("id"),
```

```
                resultado.getString("nome"),
                resultado.getString("telefone")
            );
        }

    } catch (SQLException e) {

        throw new RuntimeException(
            "Erro ao buscar cliente.",
            e
        );
    }

    return null;
}

@Override
public ArrayList<Cliente> listar() {

    ArrayList<Cliente> clientes =
            new ArrayList<>();

    String sql = """
            SELECT id, nome, telefone
            FROM cliente
            """;

    try (
            Connection conexao = Conexao.getConexao();
            PreparedStatement comando =
                    conexao.prepareStatement(sql);
            ResultSet resultado =
                    comando.executeQuery()
    ) {

        while (resultado.next()) {

            Cliente cliente = new Cliente(
                    resultado.getInt("id"),
                    resultado.getString("nome"),
                    resultado.getString("telefone")
            );

            clientes.add(cliente);
        }

    } catch (SQLException e) {

        throw new RuntimeException(
            "Erro ao listar clientes.",
            e
        );
    }

    return clientes;
}

@Override
public boolean alterar(int id, Cliente cliente) {

    String sql = """
            UPDATE cliente
            SET nome = ?, telefone = ?
```

```
            WHERE id = ?
            """;

    try (
            Connection conexao = Conexao.getConexao();
            PreparedStatement comando =
                    conexao.prepareStatement(sql)
    ) {

        comando.setString(1, cliente.getNome());
        comando.setString(2, cliente.getTelefone());
        comando.setInt(3, id);

        return comando.executeUpdate() > 0;

    } catch (SQLException e) {

        throw new RuntimeException(
                "Erro ao alterar cliente.",
                e
        );
    }
  }

  @Override
  public boolean remover(int id) {

    String sql = """
            DELETE FROM cliente
            WHERE id = ?
            """;

    try (
            Connection conexao = Conexao.getConexao();
            PreparedStatement comando =
                    conexao.prepareStatement(sql)
    ) {

        comando.setInt(1, id);

        return comando.executeUpdate() > 0;

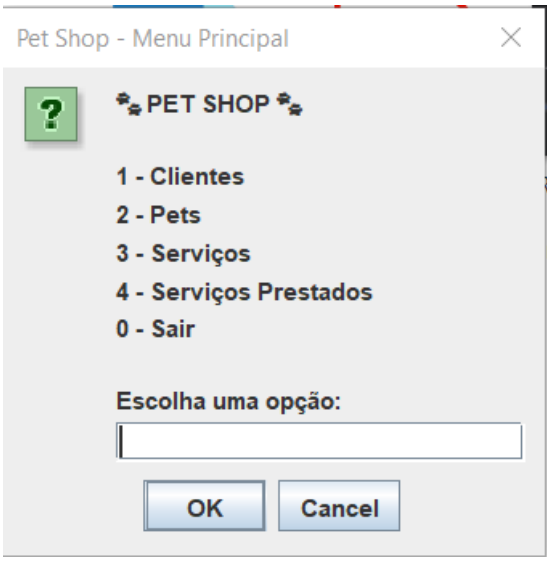
    } catch (SQLException e) {

        throw new RuntimeException(
                "Erro ao remover cliente.",
                e
        );
    }
  }
}
```

Lembre-se

Programar com o paradigma orientado a objetos, não garante fácil manutenção ou reaproveitamento de código, o programador deve usar corretamente os princípios do paradigma, junto com padrões de projeto e bons práticas da engenharia de software para usufruir dos benefícios da POO.

Tente inserir uma interface gráfica nova, veja que somente a camada de visão seria alterada, é só injetar os controllers nela:





Fim!